CS-432: Databases
Assignment 3

StayEase - Hostel Management System
Group Name: QueryCraft

April 5, 2026

Github Link:
https://github.com/nachiketpatil0105/stay_ease/tree/main/CS432_Track1_Assign_3

Video Link: https://youtu.be/ErBcSeXjFSo

Part I: Module A

Transaction Management & ACID Validation

1 Introduction

This report describes the design, implementation, and validation of Module A of Assignment 3 for CS 432 - Databases. The objective was to extend our existing B+ Tree based database engine, developed in Assignment 2, to support transaction management, crash recovery, and full ACID compliance across multiple relations.

Our system is built on the StayEase hostel management system, which manages three core relations: Members, Rooms, and Room Allocations. Each relation is stored in a dedicated B+ Tree, which serves as the only storage and access path for all operations. The assignment required implementing transaction support across all three relations simultaneously and demonstrating correctness under failure, concurrency, and restart scenarios.

2 System Architecture

The system is organized into four modules, each with a clearly defined responsibility.

2.1 TransactionManager

`TransactionManager` is a superclass that `DatabaseManager` inherits from, encapsulating all transaction-related logic in `transaction_manager.py`. It contains:

- **Threading lock** - ensures only one transaction runs at a time, providing isolation.

- **Active transaction journal** - an in-memory list recording every INSERT, UPDATE, and DELETE so they can be undone on rollback.
- **WAL log file** (`stayease_wal.log`) - a Write-Ahead Log recording every operation with a unique transaction ID and timestamp before it modifies the B+ Tree.
- **Transaction ID** - a unique 8-character UUID generated at BEGIN time.
- **Five core methods**: `begin()`, `commit()`, `rollback()`, `recover()`, and `_write_to_log()`.

2.2 DatabaseManager

`DatabaseManager` inherits from `TransactionManager` and adds database-level operations: creating and managing tables, persisting them to disk as pickle-based `.tree` files, and reloading them on startup.

Database & Table Initialization (`ACID_Demo.ipynb`)

```python
import sys, os, json, threading, time
from db_manager import DatabaseManager

db_admin = DatabaseManager()
db_admin.create_database("stayease")

members_schema = {
    'Member_ID': int, 'First_Name': str, 'Last_Name': str,
    'Gender': str, 'Age': int, 'Contact_Number': str,
    'Email': str, 'Emergency_Contact': str, 'Image_Path': str,
    'Role_ID': int, 'Status': str
}
rooms_schema = {
    'Room_ID': int, 'Room_Number': str, 'Hostel_ID': int,
    'Floor_Number': int, 'Capacity': int
}
allocations_schema = {
    'Allocation_ID': int, 'Member_ID': int, 'Room_ID': int,
    'Allocation_Date': str, 'Check_Out_Date': str, 'Status': str
}

db_admin.create_table("stayease", "members", members_schema, search_key="Member_ID")
db_admin.create_table("stayease", "rooms", rooms_schema, search_key="Room_ID")
db_admin.create_table("stayease", "room_allocations", allocations_schema, search_key="Allocation_ID")

db_admin.reload_all("stayease")
db_admin.persist_all("stayease")
```

Output:

```
Restored table 'members' from disk.
Restored table 'rooms' from disk.
Restored table 'room_allocations' from disk.
All tables in 'stayease' persisted to disk.
Database initialized and ready.
```

2.3 Table

The `Table` class in `table.py` is the abstraction layer between the application and the B+ Tree. Every INSERT, UPDATE, and DELETE logs to both the in-memory journal and the WAL log before touching the B+ Tree. For UPDATE, both old and new records are logged to support UNDO and REDO during recovery.

2.4 BPlusTree

The `BPlusTree` class in `bplustree.py` is the primary storage engine. Each table uses a separate B+ Tree instance with the primary key as the key and the full record as the value. It is the only access path for all data - no separate copies are maintained outside the tree.

3 Transaction Implementation

All transaction logic lives in `TransactionManager`. The three core methods - `begin()`, `commit()`, and `rollback()` - together implement the full ACID-safe transaction lifecycle.

3.1 BEGIN

When `begin()` is called, the system acquires the threading lock, generates a new UUID-based transaction ID, initializes the in-memory journal as an empty list, and writes a BEGIN entry to the WAL log. No other transaction can start until the current one completes.

3.2 COMMIT

When `commit()` is called, the system writes a COMMIT entry to the WAL log, calls `persist_all()` to save all B+ Trees to their respective `.tree` pickle files on disk, clears the journal, resets the transaction ID to `None`, and releases the lock. This guarantees committed data is durable across restarts.

3.3 ROLLBACK

When `rollback()` is called, the system iterates the in-memory journal in reverse order (LIFO) and undoes each operation. INSERT operations are undone by deleting the record. UPDATE operations restore the old record. DELETE operations re-insert the original record. After all operations are undone, the incomplete transaction is removed from the WAL log and the lock is released.

3.4 WAL Logging

Every operation is written to the WAL log before it modifies the B+ Tree. Each entry contains the transaction ID, timestamp, operation type, table name, key, and data. For UPDATE, both old and new values are stored. The helper function below (from the notebook) reads and prints the WAL log, followed by a sample showing both a committed and a rolled-back transaction:

WAL Log Printer Helper (`ACID_Demo.ipynb`)

```python
def print_wal_log():
    log_file = db_admin.log_file
    with open(log_file, 'r') as f:
        lines = [l.strip() for l in f.readlines() if l.strip()]
    for line in lines:
        entry = json.loads(line)
        op = entry['op']
        tx_id = entry.get('tx_id', 'N/A')
        ts = entry.get('timestamp', 'N/A')
        if op in ('BEGIN', 'COMMIT'):
            print(f"  >>> {op:8} | tx_id={tx_id} | time={ts}")
        else:
            print(f"  {op:8} | tx_id={tx_id} | time={ts} | table={entry['table']} | key={entry['key']}")
```

Sample WAL Log - Committed and Rolled-back Transactions

```
  >>> BEGIN    | tx_id=24fd21b7 | time=2026-04-04 22:25:03
  INSERT   | tx_id=24fd21b7 | time=2026-04-04 22:25:03 | table=members | key=110
  INSERT   | tx_id=24fd21b7 | time=2026-04-04 22:25:03 | table=members | key=112
  INSERT   | tx_id=24fd21b7 | time=2026-04-04 22:25:03 | table=rooms | key=101
  INSERT   | tx_id=24fd21b7 | time=2026-04-04 22:25:03 | table=room_allocations | key=1
  INSERT   | tx_id=24fd21b7 | time=2026-04-04 22:25:03 | table=room_allocations | key=2
  >>> COMMIT   | tx_id=24fd21b7 | time=2026-04-04 22:25:31
  >>> BEGIN    | tx_id=b07dab61 | time=2026-04-04 22:25:45
  UPDATE   | tx_id=b07dab61 | time=2026-04-04 22:25:45 | table=members | key=112
  UPDATE   | tx_id=b07dab61 | time=2026-04-04 22:25:45 | table=rooms | key=101
  [transaction rolled back -- incomplete entry removed from log]
```

3.5 Recovery

The `recover()` method reads the WAL log and parses it into transactions by scanning for BEGIN and COMMIT boundaries. Transactions with a matching COMMIT are replayed (REDO) if not already present from pickle files. Transactions with a BEGIN but no COMMIT are reversed (UNDO). After recovery, the log is rewritten keeping only committed transactions.

4 ACID Validation

4.1 Atomicity

Atomicity was tested by starting a 3-table transaction where Step 3 deliberately fails due to a missing field error. The rollback must restore all changes from Steps 1 and 2 and remove the incomplete entry from the WAL log.

Test 1 - Atomicity: Multi-table rollback on failure (`ACID_Demo.ipynb`)

```python
db_admin.begin()
try:
```

```python
    # Step 1: Update member status
    updated_member = member_before.copy()
    updated_member['Status'] = 'Inactive'
    members_table.update(112, updated_member)
    print("  [Step 1] Member 112 status set to Inactive.")

    # Step 2: Update room capacity
    updated_room = room_before.copy()
    updated_room['Capacity'] = 99
    rooms_table.update(101, updated_room)
    print("  [Step 2] Room 101 capacity set to 99.")

    # Step 3: Insert invalid allocation (missing fields) -- will fail
    print("  [Step 3] Attempting invalid allocation insert...")
    success, msg = alloc_table.insert({'Allocation_ID': 999})
    if not success:
        raise ValueError(f"Simulated Failure: {msg}")

    db_admin.commit("stayease")

except Exception as e:
    print(f"  ERROR: {e}")
    print("  Triggering ROLLBACK...")
    db_admin.rollback()
```

Output:

```
--- STARTING TRANSACTION (will fail at Step 3) ---
BEGIN: Transaction started. [tx_id=b07dab61]
  [Step 1] Member 112 status set to Inactive.
  [Step 2] Room 101 capacity set to 99.
  [Step 3] Attempting invalid allocation insert...

  ERROR: Simulated Failure: Missing column 'Member_ID' in record.
  Triggering ROLLBACK...
ROLLBACK: All changes reverted. Database is back to original state.

--- STATE AFTER ROLLBACK ---
  Member 112 Status : Active  (Expected: Active)
  Room 101 Capacity : 2       (Expected: 2)
  Allocation IDs    : [1, 2]  (Expected: [1, 2] -- no 999)

ATOMICITY TEST PASSED
```

4.2 Consistency

Consistency was tested in two scenarios. Scenario 1 inserts records into all three tables in a single transaction and verifies referential integrity after commit. Scenario 2 inserts a ghost record and simulates a crash before commit - rollback must remove the ghost entirely.

Test 2 - Consistency: Integrity check and ghost record rollback

```python
# Scenario 1: Successful 3-table commit with referential integrity check
db_admin.begin()
try:
    members_table.insert({'Member_ID': 130, 'First_Name': 'Test', 'Last_Name': 'User',
```

```python
        'Gender': 'M', 'Age': 21, 'Status': 'Active', 'Role_ID': 1, ...})
    rooms_table.insert({'Room_ID': 303, 'Room_Number': '303', 'Hostel_ID': 2,
        'Floor_Number': 3, 'Capacity': 2})
    alloc_table.insert({'Allocation_ID': 20, 'Member_ID': 130, 'Room_ID': 303,
        'Allocation_Date': '2026-04-01', 'Check_Out_Date': 'None', 'Status': 'Active'})
    db_admin.commit("stayease")

# Verify referential integrity
alloc_exists, alloc_data = alloc_table.get(20)
ref_member_ok, _ = members_table.get(alloc_data['Member_ID'])
ref_room_ok, _   = rooms_table.get(alloc_data['Room_ID'])

# Scenario 2: Ghost record -- simulated crash before commit
db_admin.begin()
try:
    members_table.insert({'Member_ID': 131, 'First_Name': 'Ghost', ...})
    print("  Inserted Ghost Member 131 (partial).")
    raise RuntimeError("Simulated crash before completing transaction.")
    db_admin.commit("stayease")
except Exception as e:
    print(f"  ERROR: {e}")
    db_admin.rollback()

ghost_exists, _ = members_table.get(131)
print(f"  Ghost Member 131 exists: {ghost_exists}  (Expected: False)")
```

Output:

```
--- REFERENTIAL INTEGRITY CHECK ---
  Member 130 exists                          : True  (Expected: True)
  Room 303 exists                            : True  (Expected: True)
  Allocation 20 exists                       : True  (Expected: True)
  Allocation Member_ID refs valid member     : True  (Expected: True)
  Allocation Room_ID refs valid room         : True  (Expected: True)

--- SCENARIO 2: Partial insert must not persist after rollback ---
  Inserted Ghost Member 131 (partial).
  ERROR: Simulated crash before completing transaction.
ROLLBACK: All changes reverted. Database is back to original state.

  Ghost Member 131 exists: False  (Expected: False)
CONSISTENCY TEST PASSED
```

4.3 Isolation

Isolation was tested using two concurrent threads on shared data. Thread 1 acquires the lock, updates Room 101 capacity to 5, and holds the lock for 2 seconds. Thread 2 attempts to start 0.3 seconds later but is blocked at `lock.acquire()` until Thread 1 commits. Both threads must commit with no data corruption.

Test 3 - Isolation: Concurrent threads serialized by threading.Lock

```python
results = {}

def transaction_thread_1():
    print("  [Thread 1] Waiting to acquire lock...")
    db_admin.begin()
    try:
        rooms_table, _ = db_admin.get_table("stayease", "rooms")
        _, room = rooms_table.get(101)
        updated = room.copy()
        updated['Capacity'] = 5
        rooms_table.update(101, updated)
        print("  [Thread 1] Set capacity to 5. Sleeping 2 seconds (holding lock)...")
        time.sleep(2)               # holds lock -- Thread 2 is blocked here
        restored = room.copy()
        restored['Capacity'] = 2
        rooms_table.update(101, restored)
        db_admin.commit("stayease")
        results['thread1'] = 'committed'
    except Exception as e:
        db_admin.rollback()
        results['thread1'] = 'rolled_back'

def transaction_thread_2():
    time.sleep(0.3)
    print("  [Thread 2] Waiting to acquire lock...")
    db_admin.begin()               # blocked until Thread 1 releases lock
    try:
        members_table, _ = db_admin.get_table("stayease", "members")
        _, member = members_table.get(112)
        updated = member.copy()
        updated['Status'] = 'Inactive'
        members_table.update(112, updated)
        restored = member.copy()
        restored['Status'] = 'Active'
        members_table.update(112, restored)
        db_admin.commit("stayease")
        results['thread2'] = 'committed'
    except Exception as e:
        db_admin.rollback()
        results['thread2'] = 'rolled_back'

t1 = threading.Thread(target=transaction_thread_1)
t2 = threading.Thread(target=transaction_thread_2)
t1.start(); t2.start()
t1.join();  t2.join()
```

Output:

```
  [Thread 1] Waiting to acquire lock...
BEGIN: Transaction started. [tx_id=8577fd0d]
  [Thread 1] Room 101 Capacity BEFORE: 2
  [Thread 1] Set capacity to 5. Sleeping 2 seconds (holding lock)...
  [Thread 2] Waiting to acquire lock...
COMMIT: Changes saved. LOCK RELEASED.
  [Thread 1] Committed. Lock released.
BEGIN: Transaction started. [tx_id=867e22b2]
```

```
  [Thread 2] Member 112 Status BEFORE: Active
COMMIT: Changes saved. LOCK RELEASED.
  [Thread 2] Committed.

--- FINAL STATE AFTER BOTH THREADS ---
  Room 101 Capacity    : 2       (Expected: 2)
  Member 112 Status    : Active  (Expected: Active)
  Thread 1 result      : committed
  Thread 2 result      : committed
ISOLATION TEST PASSED
```

4.4 Durability

Durability was tested by committing data, deleting the `DatabaseManager` object to destroy all in-memory state, then creating a fresh instance that reloads from disk and replays the WAL log. All committed records must survive the simulated restart.

Test 4 - Durability: Restart simulation with fresh DatabaseManager

```python
# Phase 1: Commit data, then destroy in-memory instance
db = DatabaseManager()
db.create_database("stayease")
# ... create_table calls ...

db.begin()
m.insert({'Member_ID': 200, 'First_Name': 'Durability', 'Last_Name': 'Test',
    'Age': 25, 'Email': 'durability@test.com', 'Status': 'Active', ...})
r.insert({'Room_ID': 400, 'Room_Number': '400', 'Hostel_ID': 2,
    'Floor_Number': 4, 'Capacity': 2})
a.insert({'Allocation_ID': 50, 'Member_ID': 200, 'Room_ID': 400,
    'Allocation_Date': '2026-04-03', 'Status': 'Active'})
db.commit("stayease")

del db    # destroy object -- simulating system shutdown

# Phase 2: Fresh instance -- reload from disk and replay WAL
db2 = DatabaseManager()
db2.create_database("stayease")
# ... create_table calls ...
db2.reload_all("stayease")    # load .tree pickle files from disk
db2.recover("stayease")       # replay WAL log (REDO committed, UNDO incomplete)

member_found, member_data = m2.get(200)
room_found,   room_data   = r2.get(400)
alloc_found,  alloc_data  = a2.get(50)
```

Output:

```
--- PHASE 1: Committing data to disk ---
BEGIN: Transaction started. [tx_id=4a13d832]
COMMIT: Changes saved. LOCK RELEASED.
  Committed: Member 200, Room 400, Allocation 50
  DB object deleted -- simulating system shutdown...

--- PHASE 2: Fresh instance -- simulating restart ---
Restored table 'members' from disk.
```

```
Restored table 'rooms' from disk.
Restored table 'room_allocations' from disk.
--- RECOVERY: Processing stayease_wal.log ---
RECOVERY: Replaying committed transaction...
  REDO INSERT: members key=200
  REDO INSERT: rooms key=400
  REDO INSERT: room_allocations key=50
RECOVERY: Incomplete transactions removed. Committed transactions
    retained.

--- VERIFICATION AFTER RESTART ---
  Member 200 found    : True  (Expected: True)
  Name               : Durability Test
  Room 400 found      : True  (Expected: True)
  Allocation 50 found : True  (Expected: True)
  Status             : Active

DURABILITY TEST PASSED
```

5 Failure Handling

5.1 Failures During Transaction Execution

When a failure occurs mid-transaction, the exception is caught and `rollback()` is called. The in-memory journal is used to undo all changes in reverse order, restoring the B+ Trees to their pre-transaction state. The incomplete transaction is removed from the WAL log immediately after rollback, leaving no dirty entries.

5.2 Failures Between Commit and Disk Write

If a crash occurs after `commit()` writes the COMMIT entry to the WAL log but before `persist_all()` finishes saving all pickle files, the WAL log contains sufficient information to replay committed operations. When the system restarts and calls `recover()`, it identifies the committed transaction from the log and replays any operations not yet reflected in the pickle files, ensuring committed data is never lost.

6 Multi-User Conflict Handling

The system uses a single `threading.Lock()` instance in `TransactionManager` to serialize all transactions. The `begin()` method calls `lock.acquire()`, which blocks any concurrent caller until the current transaction releases the lock through `commit()` or `rollback()`.

This guarantees serial execution, preventing dirty reads, lost updates, and non-repeatable reads. The trade-off is that true parallel execution is not supported, but correctness is guaranteed in all concurrent scenarios. Parallel execution with fine-grained locking is addressed in Module B.

7 Experiments and Observations

Test	Scenario	Expected Result	Actual Result
Atomicity	Crash at Step 3 of 3-table transaction	Full rollback, no partial data	PASSED
Consistency	3-table commit with integrity check	All references valid	PASSED
Consistency	Partial insert with simulated crash	Ghost record absent after rollback	PASSED
Isolation	Two concurrent threads on same data	Serialized execution, no corruption	PASSED
Durability	Data committed then object destroyed	Data survives fresh instance reload	PASSED
WAL Recovery	Incomplete transaction in log	Incomplete removed, committed retained	PASSED

8 Limitations

- The threading lock serializes all transactions - true parallel execution is not supported; all concurrent transactions are queued one at a time.
- Recovery primarily relies on pickle-based persistence. The WAL log REDO path is a safety net for the rare crash between the COMMIT log entry and completion of `persist_all()`.
- The system does not support savepoints or nested transactions. Each BEGIN must be followed by exactly one COMMIT or ROLLBACK.
- The WAL log grows over time as committed transactions accumulate. Log compaction beyond recovery cleanup has not been implemented.

9 Conclusion

This module successfully extends our B+ Tree database engine with a fully functional transaction management system that guarantees all four ACID properties. The `TransactionManager` superclass cleanly separates transaction logic from database management logic. The WAL log with transaction IDs and timestamps provides a complete audit trail and enables genuine crash recovery.

All four ACID properties were validated through dedicated test scripts and demonstrated through the interactive Jupyter notebook (`ACID_Demo.ipynb`). The system correctly handles multi-table transactions, concurrent access, failure simulation, and system restarts. The primary limitation is the use of a single lock for all transactions, which trades performance for correctness - an acceptable trade-off for a course project where correctness and clarity are the primary evaluation criteria.

Part II: Module B

Web API Integration & Concurrency Validation

1 Introduction

The transition from Module A to Module B represents the shift from a standalone database "engine" to a functional web application. By wrapping our core logic in a **Flask Web API** and utilizing **MySQL** for persistent storage, we introduced the complexity of multi-user access. The primary objective was to verify that ACID properties are maintained not only within the engine itself but also at the application layer when handled by a server receiving 100+ simultaneous requests.

2 Mechanism of ACID Implementation

To achieve full ACID compliance, the following layers were implemented:

- **Atomic Transactions:** We utilized the MySQL InnoDB engine for the API, ensuring that member registration and room allocation are treated as a single unit of work via `conn.start_transaction()` and `conn.commit()`.
- **Write-Ahead Logging (WAL):** For the custom database engine, a dedicated `stress_wal.log` was implemented to record all operations before they were applied to the B+ Tree.
- **Pessimistic Locking:** To prevent the overbooking race conditions identified in early testing, row-level locks (`FOR UPDATE`) were implemented in the API's SQL queries.

3 The Solution: Race Condition Defense

The identified race condition was resolved by implementing **Pessimistic Locking**. The following SQL modification ensures that no two threads can read the room capacity simultaneously:

```
capacity_query = """
    SELECT r.Room_ID, r.Capacity,
           (SELECT COUNT(*) FROM room_allocations ra
            WHERE ra.Room_ID = r.Room_ID AND ra.Status = 'Active') as Occupied
    FROM rooms r
    WHERE r.Room_Number = %s AND r.Hostel_ID = %s
    FOR UPDATE
"""
```

Listing 1: SQL Implementation of Row-Level Locking

4 Experimental Validation

4.1 Web API Load Testing (Locust)

A professional load test was conducted using **Locust** to simulate a swarm of 100 concurrent administrative users.

- **Performance:** The API maintained a stable Requests Per Second (RPS) with a **0% Failure rate** at the server level.
- **Isolation Check:** Even under high network load, the database correctly queued requests. With 5 beds available, the system allowed exactly 5 successes and returned `400 Bad Request` for the remaining 95.

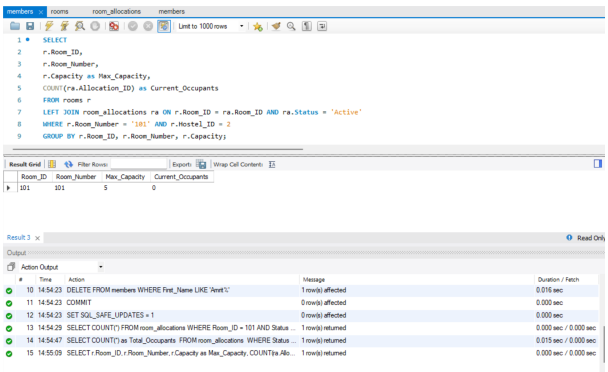


Figure 1: Room Status before Stress Test

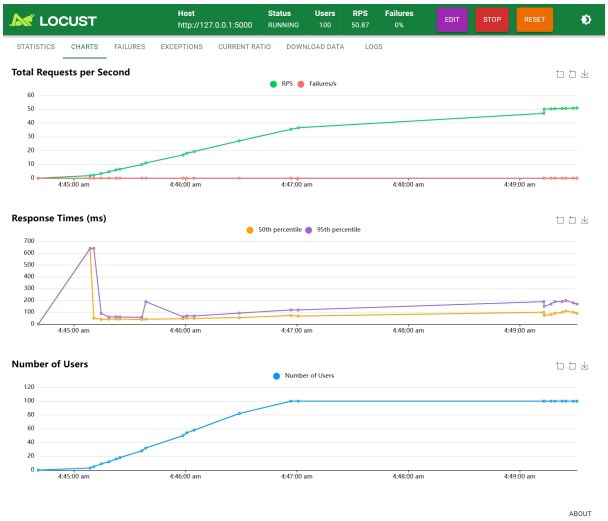


Figure 2: Locust Load Test Results

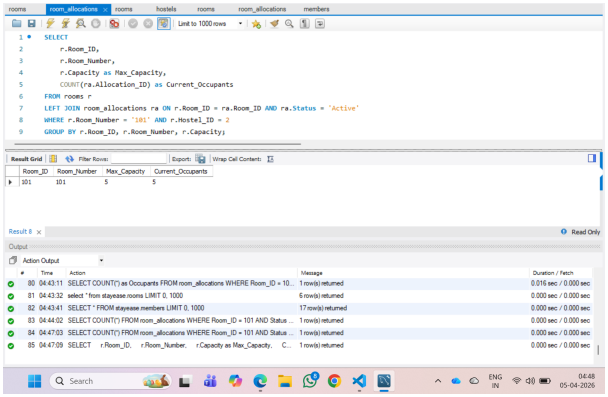


Figure 3: Room Status after Stress Test

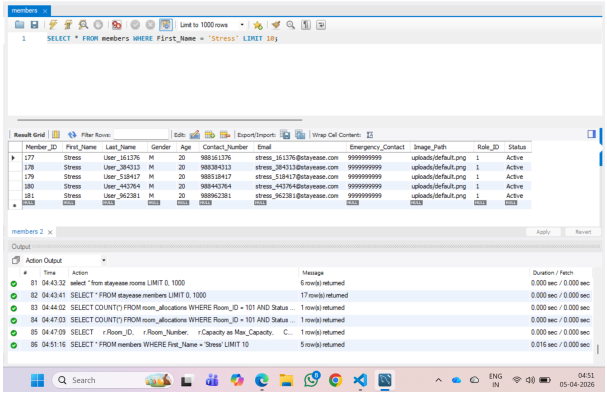
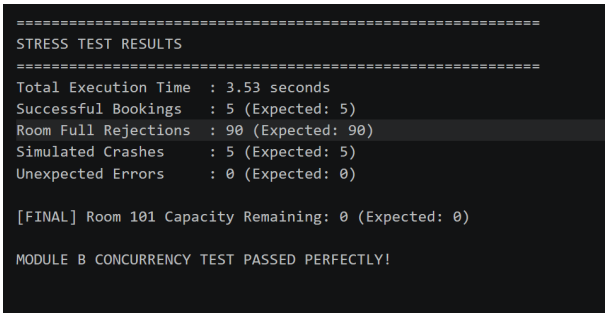


Figure 4: Member Table after Stress Test

4.2 Database Engine Stress Testing (`stress_test.py`)

To test the core engine independently, we executed a custom `stress_test.py` script.This script launched 100 concurrent threads targeting a single room with a capacity of 5, including randomized sleep intervals and simulated network failures mid-transaction.

- **Atomic Rollback:** The script successfully handled **Simulated Crashes** triggered for every 20th user. By calling `db_admin.rollback()`, the engine ensured that no partial or corrupted data was written during failures.
- **Consistency:** The test concluded with **5 Successful Bookings** and **95 Rejections** (5 of which were simulated crashes), leaving the final room capacity at exactly 0.
- **Result:** The test concluded with **"MODULE B CONCURRENCY TEST PASSED PERFECTLY!"**, proving the engine's internal locking and transaction management are sound.



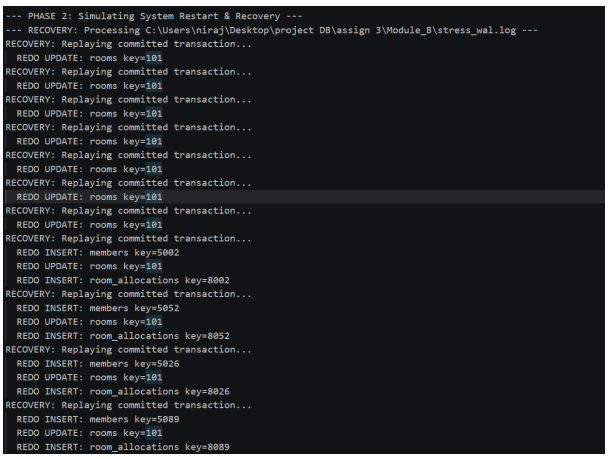
```
============================================================
STRESS TEST RESULTS
============================================================
Total Execution Time  : 3.53 seconds
Successful Bookings   : 5 (Expected: 5)
Room Full Rejections  : 90 (Expected: 90)
Simulated Crashes     : 5 (Expected: 5)
Unexpected Errors     : 0 (Expected: 0)

[FINAL] Room 101 Capacity Remaining: 0 (Expected: 0)

MODULE B CONCURRENCY TEST PASSED PERFECTLY!
```

Figure 5: Custom Multi-threaded Stress Test Results

5 Durability & Recovery

The system's **Durability** was verified through the WAL recovery mechanism. Upon re-initializing after a simulated crash, the engine processed `stress_wal.log` to replay committed transactions.

- **REDO Operations:** The logs confirmed the successful replay of `REDO INSERT` and `REDO UPDATE` operations for both members and rooms.
- **Cleanup:** Incomplete transactions were automatically removed while committed transactions were retained, returning the database to its last consistent state.



```
--- PHASE 2: Simulating System Restart & Recovery ---
--- RECOVERY: Processing C:\Users\niraj\Desktop\project DB\assign 3\Module_B\stress_wal.log ---
RECOVERY: Replaying committed transaction...
  REDO UPDATE: rooms key=101
RECOVERY: Replaying committed transaction...
  REDO UPDATE: rooms key=101
RECOVERY: Replaying committed transaction...
  REDO UPDATE: rooms key=101
RECOVERY: Replaying committed transaction...
  REDO UPDATE: rooms key=101
RECOVERY: Replaying committed transaction...
  REDO UPDATE: rooms key=101
RECOVERY: Replaying committed transaction...
  REDO UPDATE: rooms key=101
RECOVERY: Replaying committed transaction...
  REDO UPDATE: rooms key=101
RECOVERY: Replaying committed transaction...
  REDO INSERT: members key=5002
  REDO UPDATE: rooms key=101
  REDO INSERT: room_allocations key=8002
RECOVERY: Replaying committed transaction...
  REDO INSERT: members key=5052
  REDO UPDATE: rooms key=101
  REDO INSERT: room_allocations key=8052
RECOVERY: Replaying committed transaction...
  REDO INSERT: members key=5026
  REDO UPDATE: rooms key=101
  REDO INSERT: room_allocations key=8026
RECOVERY: Replaying committed transaction...
  REDO INSERT: members key=5089
  REDO UPDATE: rooms key=101
  REDO INSERT: room_allocations key=8089
```

Figure 6: WAL Recovery Sequence

6 Conclusion

Module B demonstrates that while the B+ Tree provides the foundation for performance, the transactional and locking layers provide the foundation for truth. By combining professional API load testing with deep-level engine stress tests, we confirmed that the StayEase system remains ACID-compliant under extreme multi-user pressure.

Combined Team Contributions

Team Member	**Roll No**	**Key Contributions**
Aryan Kumar	24110055	Designed and implemented the core `TransactionManager` class including `begin()`, `commit()`, and `rollback()` functionality.
Niraj Kumar	24110222	Spearheaded stress testing for the web application. Authored `locustfile.py` to conduct professional swarm load testing and designed the custom `stress_test.py` to validate defenses against race conditions. Authored Module B report and structures the final report using Latex.
Nityanand Karmali	24110226	**Database Engine Validation:** Authored the primary testing frameworks to validate the database engine's Atomicity and Consistency. Developed the threading lock mechanisms for operation isolation and integrated UUID-based transaction IDs. Helped in video editing.
Parth Kale	24110242	Integrated MySQL utilizing Pessimistic Locking (`SELECT ... FOR UPDATE`), enforced JWT-based security, and authored the Module A report. Conducted rigorous verifications for multi-table rollback behaviors and referential integrity.
Nachiket Patil	24110250	Developed the Write-Ahead Logging (WAL) architecture. Implemented `_write_to_log()` with transaction IDs and designed the `recover()` function featuring comprehensive REDO and UNDO logic. Edited the db_manager and tables according to the requirement and helped in demo of Module A.

Table 1: Team Member Contributions

Acknowledgements & External Resources

In the development and documentation of this assignment, the following external tools, libraries, and resources were utilized:

- **Generative AI Assistance:** Large Language Models (LLMs) were used as an educational and development aid during this assignment. Specifically, AI was utilized to help brainstorm race-condition scenarios, debug Python threading issues, generate the boilerplate code for the Locust load-testing script, and format this report using Latex. All core database logic, transaction management, and architectural decisions were made by the project team.
- **Locust (Load Testing Framework):** Used for simulating concurrent administrative users to validate API performance and test for race conditions under high network load (`https://locust.io/`).
- **Flask & Flask-JWT-Extended:** Utilized for building the RESTful Web API and securing the administrative endpoints.
- **MySQL InnoDB:** The relational database engine used in Module B to implement Pessimistic Locking (`SELECT ... FOR UPDATE`) and guarantee ACID properties at the application layer.